Projekt nr 7: Bajtolandia

Algorytmy i Struktury Danych

Autorzy:
Karolina Gralak
Wiktor Chmiel

11 grudnia 2025

Wydział Matematyki i Nauk Informacyjnych
POLITECHNIKA WARSZAWSKA

Spis treści

1 Wprowadzenie **4**

1.1 Cel projektu 4

1.2 Treść zadania 4

2 Opis problemu **5**

2.1 Model matematyczny 5

2.2 Reprezentacja grafowa 6

3 Opis algorytmu **7**

3.1 Wybór metody 7

3.2 Algorytm Dijkstry 7

4 Rozwiązania techniczne i implementacja **11**

4.1 Struktura klas 11

5 Testy i weryfikacja działania **13**

5.1 Interfejs użytkownika 13

5.2 Opis trybów działania 13

5.3 Scenariusze testowe 14

5.3.1 Test 1: Tryb ręczny - walidacja i omijanie przeszkód 15

5.3.2 Test 2: Weryfikacja poprawności na losowej mapie (6×11) 16

5.3.3 Test 3: Omijanie bariery kosztów 18

5.4 Wnioski z testów 19

6 Podsumowanie i wnioski **20**

7 Podział pracy w zespole **20**

1 Wprowadzenie

1.1 Cel projektu

Celem projektu było zaprojektowanie, zaimplementowanie oraz przeprowadzenie szczegółowej analizy algorytmu rozwiązującego problem optymalizacyjny w regionie Bajtolandia. Zadanie polegało na znalezieniu najtańszej trasy modernizacji infrastruktury drogowej łączącej dwa miasta: Bajtogórę oraz Bitogórę. Problem ten można sprowadzić do problemu poszukiwania najkrótszej ścieżki w grafie ważonym. Specyfika zadania polegała na tym, że wagi (koszty) przypisane były do węzłów (subregionów), a nie do krawędzi, co wymagało odpowiedniego dostosowania standardowych algorytmów grafowych.

W ramach prac projektowych:

- Opracowano model matematyczny problemu.
- Zaimplementowano rozwiązanie w języku Python.
- Przeprowadzono analizę teoretyczną złożoności obliczeniowej.
- Wykonano testy weryfikujące poprawność i wydajność rozwiązania.

1.2 Treść zadania

Bajtolandia to szczególnie urokliwy region Bajtocji, ostatnio coraz bardziej popularny wśród turystów. Mapę tego regionu można w uproszczeniu przedstawić jako prostokąt $n \times m$ złożony z $n \cdot m$ pól (subregionów), przy czym $0 \leq n, m \leq 100$. Z subregionem (x, y) sąsiadują subregiony $(x+1, y)$, $(x-1, y)$, $(x, y+1)$ oraz $(x, y-1)$, o ile nie jest to subregion brzegowy (czyli jeśli $x = 0$, $x = n$, $y = 0$, $y = m$, te regiony mają oczywiście mniej subregionów sąsiadujących). Każde dwa sąsiadujące subregiony są połączone drogami, nie ma bezpośrednich połączeń drogowych między subregionami, które nie sąsiadują.

Szczególną popularnością cieszą się ostatnio dwa miasteczka, Bajtogród i Bitogór, położone w dwóch różnych subregionach, które nie sąsiadują. Niestety infrastruktura drogowa regionu jest w niezbyt dobrym stanie i wymaga inwestycji. Dla każdego z nich znany jest koszt modernizacji dróg. Władze Bajtolandii postanowiły najpierw zmodernizować drogi między Bitogórem a Bajtogrodem. Niestety fundusze są ograniczone. Bajtazarowi zlecono wyznaczenie trasy rewolwerowej między tymi miastami wylotami przez przylegające subregiony tak, by koszt łącznej modernizacji był najniższy. Brazazar ma kłopot, gdyż nie zna się na algorytmice i, znając Wasze umiejętności, zwrócił się do Was o pomoc.

Wejście

Losowo (bądź z klawiatury) należy podać:

1. liczby $0 \leq n, m \leq 100$ oznaczające obszar Bajtolandii,
2. koszty $k_{i,j}$, $i = 1, \ldots, n$, $j = 1, \ldots, m$, modernizacji dróg w subregionie położonym w polu (i, j).

Wyjście

Wynikiem jest ciąg pól odpowiadających subregionom, których drogi należy zmodernizować i minimalizujących łączny koszt ich modernizacji. Kolejne pola w tym ciągu to sąsiadujące subregiony, przez które ma prowadzić poszukiwana trasa.

2 Opis problemu

W tym rozdziale przedstawiamy formalną definicję problemu, wprowadzamy niezbędne oznaczenia matematyczne oraz opisujemy transformację mapy terenu na strukturę grafową.

2.1 Model matematyczny

Niech M będzie macierzą o wymiarach $n \times m$, reprezentującą mapę Bajtolandii. Element macierzy $M[i][j]$ (gdzie $0 \leq i \leq n, 0 \leq j \leq m$) przechowuje wartość kosztu modernizacji danego subregionu:

$$k_{i,j} \in \mathbb{N} \cup \{0\}$$

Zdefiniujmy relację sąsiedztwa S. Dwa pola (x_1, y_1) oraz (x_2, y_2) są sąsiadami wtedy i tylko wtedy, gdy:

$$|x_1 - x_2| + |y_1 - y_2| = 1$$

Jest to tak zwana odległość miejska (Manhattan distance).

Naszym celem jest znalezienie ciągu pól $P = (p_1, p_2, \ldots, p_k)$, gdzie $p_r = (x_r, y_r)$, spełniającego warunki:

1. $p_1 = \text{Start (Bajtogóra)}$,
2. $p_k = \text{Cel (Bitogóra)}$,
3. $\forall_{1 \leq r < k}\ p_r$ sąsiaduje z p_{r+1}.

Funkcja celu, którą minimalizujemy, to suma kosztów pól na ścieżce:

$$\text{Cost}(P) = \sum_{r=1}^{k} k_{x_r, y_r} \to \min$$

2.2 Reprezentacja grafowa

Aby zastosować znane algorytmy optymalizacyjne, problem mapy przekształcamy w graf ważony $G = (V, E)$.

Wierzchołki

Zbiór wierzchołków V odpowiada wszystkim polom na mapie. Liczba wierzchołków wynosi:

$$|V| = n \cdot m$$

Dla ułatwienia implementacji, każde pole (i, j) mapujemy na unikalny identyfikator liczbowy $ID \in \{0, \ldots, nm - 1\}$ wzorem:

$$ID(i, j) = i \cdot m + j$$

gdzie i, j są indeksowane od 0.

Krawędzie

Zbiór krawędzi E tworzą pary wierzchołków, które ze sobą sąsiadują na planszy.

Wagi

W klasycznym problemie najkrótszej ścieżki wagi znajdują się na krawędziach. W naszym przypadku koszty są na wierzchołkach. Transformacja polega na przyjęciu, że waga

krawędzi skierowanej $u \to v$ jest równa kosztowi wierzchołka docelowego v.

$$w(u, v) = \text{koszt}(v)$$

Dzięki temu, wchodząc do wierzchołka v, "płacimy" jego koszt. Należy pamiętać, aby do ostatecznego wyniku dodać koszt wierzchołka startowego, który nie jest uwzględniany przez sumę wag krawędzi (ponieważ do startu nie "wchodzimy" żadną krawędzią).

3 Opis algorytmu

3.1 Wybór metody

Ponieważ koszty modernizacji są liczbami nieujemnymi ($k \geq 0$), problem sprowadza się do znalezienia najkrótszej ścieżki w grafie z nieujemnymi wagami krawędzi.

W takim przypadku najefektywniejszym i standardowym rozwiązaniem jest **algorytm Dijkstry**.

3.2 Algorytm Dijkstry

Struktury danych

- **Tablica odległości ($dist$):** Tablica o rozmiarze $n \cdot m$, inicjalizowana wartością ∞. Element $dist[v]$ przechowuje aktualnie najlepszy znany koszt dotarcia ze startowego miasta (Bajtogóry) do wierzchołka v.
- **Tablica poprzedników ($prev$):** Tablica o rozmiarze $n \cdot m$, w której $prev[v]$ zapisuje wierzchołek, z którego przychodzimy do v na najtańszej znalezionej ścieżce. Używana do rekonstrukcji końcowej trasy.
- **Zbiór przetworzonych wierzchołków (S):** Zbiór tych wierzchołków, dla których znamy minimalny koszt dotarcia. Początkowo $S = \emptyset$.

Pseudokod

Algorithm 1 Algorytm Dijkstry

```
begin
for all v ∈ V do
    D[v].dist := l(s, v)
end for
S := {s}
while S ≠ V do
    {wyznaczenie z V \ S wierzchołka najbliższego s}
    mindist := +∞
    for all w ∈ V \ S do
        if D[w].dist < mindist then
            v := w
            mindist := D[w].dist
        end if
    end for
    S := S ∪ {v}
    for all w ∈ V \ S do
        if D[w].dist > mindist + l(v, w) then
            D[w].dist := mindist + l(v, w)
            D[w].prev := v
        end if
    end for
end while
end
```

Przebieg działania

Inicjalizacja: Dla wszystkich v ustawiamy:

$$dist[v] = \infty, \qquad prev[v] = -1.$$

Niech S będzie identyfikatorem wierzchołka startowego (Bajtogóry). Ponieważ koszt wejścia na pole jest równy kosztowi jego modernizacji, ustawiamy:

$$dist[S] = cost(S).$$

Pętla główna: Powtarzamy dopóki S nie zawiera wszystkich wierzchołków:

1. **Wybór nieprzetworzonego wierzchołka o najmniejszym koszcie.**
   Szukamy wierzchołka $u \in V \setminus S$, który minimalizuje $dist[u]$. Jest to liniowe skanowanie wszystkich nieprzetworzonych wierzchołków.

2. **Dodanie wierzchołka do zbioru przetworzonych:**

$$S := S \cup \{u\}.$$

3. **Relaksacja sąsiadów wierzchołka u:** Dla każdego sąsiada v (góra, dół, lewo, prawo):

$$alt = dist[u] + cost(v).$$

   Jeżeli $alt < dist[v]$, to aktualizujemy:

$$dist[v] := alt, \qquad prev[v] := u.$$

4. **Rekonstrukcja trasy:** Startując od wierzchołka końcowego T (Bitogóry), cofamy się po tablicy $prev$ aż do wierzchołka startowego. Otrzymaną sekwencję odwracamy, uzyskując najtańszą ścieżkę.

Złożoność obliczeniowa

Niech $G = (V, E)$ będzie grafem o nieujemnych wagach krawędzi. Oznaczmy $|V| = n$. Analizujemy złożoność czasową krok po kroku.

- Pętla główna wykonywana jest $n - 1$ razy, ponieważ w każdej iteracji do zbioru S dodawany jest jeden wierzchołek.
- W każdej iteracji wykonywane jest wyszukiwanie minimum wśród wierzchołków $V \setminus S$ (pętla w linii (9)). Ponieważ rozmiar tego zbioru wynosi $n - i$ w i-tej iteracji, koszt wyszukania minimum jest rzędu $\mathcal{O}(n - i)$.
- Pętla w linii (16) obraca się n-i-1 razy,
- Po wyborze wierzchołka u wykonywana jest relaksacja wszystkich jego sąsiadów. W przypadku reprezentacji listowej koszt tego kroku jest stały względem n.

Całkowitą złożoność czasową można więc zapisać jako:

$$T(n) = \sum_{i=1}^{n-1} C(n - i + n - i - 1) = (n - 1)^2.$$

Stąd klasyczny algorytm Dijkstry ma złożoność:

$$\boxed{T(n) = \Theta(n^2)}.$$

W przypadku grafu kratowego o wymiarach $n \times m$ liczba wierzchołków wynosi $N = n \cdot m$, zatem złożoność algorytmu w projekcie wynosi:

$$T(N) = \Theta(N^2) = \Theta\big((nm)^2\big).$$

Dowód poprawności algorytmu Dijkstry

Przedstawiamy dowód poprawności klasycznej wersji algorytmu Dijkstry, oparty na indukcji względem liczności zbioru przetworzonych wierzchołków S. Algorytm zakłada, że wszystkie wagi krawędzi są nieujemne.

Krok bazowy. Niech $|S| = 1$, czyli $S = \{s\}$, gdzie s jest wierzchołkiem startowym. Najkrótsza ścieżka z s do s ma długość 0, a dla każdego innego wierzchołka $v \neq s$ początkowa wartość $D[v].dist$ równa jest wadze bezpośredniej krawędzi (s, v) (jeśli taka istnieje) albo $+\infty$ w przeciwnym razie. Inicjalizacja tablicy $dist$ jest więc poprawna.

Krok indukcyjny. Załóżmy, że dla pewnego $k \geq 1$ zbiory S oraz wartości $D[v].dist$ są poprawne dla $|S| = k$. W kolejnej iteracji algorytm wybiera wierzchołek $v \in V \setminus S$ o najmniejszej wartości $D[v].dist$. Pokażemy, że ścieżka prowadząca z s do v wyłącznie przez wierzchołki ze zbioru S jest najkrótszą możliwą ścieżką.

Niech dla sprzeczności istnieje ścieżka π' krótsza od π, która prowadzi z s do v i przechodzi przez co najmniej jeden wierzchołek spoza S. Niech w będzie pierwszym wierzchołkiem spoza S na tej ścieżce. Wówczas:

$$dist_{\pi'}(s, w) = dist_{\pi}(s, w) + l(v, w) < dist_{\pi}(s, v),$$

co oznaczałoby, że w ma mniejszą odległość od s niż v, czyli powinien być wybrany przez algorytm wcześniej jako najbliższy nieprzetworzony wierzchołek. Jest to sprzeczne z założeniem, że algorytm wybrał v.

Wniosek Wybór wierzchołka v o minimalnej wartości $D[v].dist$ jest poprawny, a relaksacja krawędzi wychodzących z v zachowuje poprawność tablicy $dist$. Indukcyjnie otrzymujemy, że po zakończeniu działania algorytmu:

$$D[v].dist = \text{długość najkrótszej ścieżki z } s \text{ do } v$$

dla każdego wierzchołka $v \in V$. Zatem algorytm Dijkstry jest poprawny.

4 Rozwiązania techniczne i implementacja

Projekt został zaimplementowany w języku **Python**.

4.1 Struktura klas

Kod programu został podzielony na dwie główne klasy:

Klasa `Graph`

Jest to klasa pomocnicza używana do tworzenia grafu na podstawie mapy Bajtolandii. Klasa ta pełni kluczową rolę w dalszych etapach projektu, ponieważ dostarcza wygodnej i przejrzystej reprezentacji grafu, na której może operować algorytm Dijkstry. Oddzielenie logiki grafowej od logiki zadania zwiększa czytelność i modularność kodu, a także upraszcza implementację wyszukiwania trasy o minimalnym koszcie.

Klasa `Bajtolandia`

Klasa `Bajtolandia` pełni rolę głównego modułu zarządzającego logiką całego zadania. Odpowiada za konstrukcję mapy Bajtolandii, przechowywanie kosztów pól, generowanie lub wczytywanie pozycji miast, tworzenie reprezentacji grafowej oraz wyznaczanie najtańszej trasy pomiędzy Bajtogórą a Bitogórą. Stanowi ona warstwę łączącą dane wejściowe z algorytmem wyszukiwania najkrótszej ścieżki.

Konstruowanie mapy. Obiekt klasy może zostać zainicjalizowany na kilka sposobów: poprzez podanie wymiarów mapy, przekazanie gotowej tablicy kosztów lub wygenerowanie

losowej planszy w zadanym zakresie. Na podstawie tych danych tworzona jest macierz `cost`, gdzie `cost[i][j]` określa koszt modernizacji pola w wierszu i i kolumnie j. Klasa wspiera również podanie z góry położenia miast, a w przeciwnym wypadku lokalizacje startu i celu mogą zostać wylosowane przy zachowaniu warunku minimalnej odległości Manhattan.

Walidacja położenia miast. Metoda `_waliduj_miasta` sprawdza poprawność pozycji przekazanych do konstruktora. Upewnia się ona, że miasta nie leżą poza granicami planszy, nie pokrywają się oraz nie sąsiadują ze sobą. Zapewnia to zgodność danych wejściowych z wymaganiami zadania.

Reprezentacja grafowa. Kluczowym etapem działania klasy jest budowa grafu na podstawie planszy, realizowana przez metodę `build_graph`. Każde pole planszy jest odwzorowane jako wierzchołek grafu, a połączenia między sąsiednimi polami (góra, dół, lewo, prawo) tworzą skierowane krawędzie o wagach odpowiadających kosztowi wejścia na pole docelowe. Generowany graf jest wykorzystywany przez algorytm Dijkstry.

Wyszukiwanie najtańszej trasy. Główny algorytm wyszukiwania ścieżki implementowany jest w metodzie `dijkstra`. Wykorzystuje ona klasyczną wersję algorytmu Dijkstry. Algorytm przechowuje tablice *dist* (minimalne koszty dotarcia) oraz *prev* (poprzednicy na najkrótszej ścieżce), a następnie, iteracyjnie wybierając nieodwiedzony wierzchołek o najmniejszym koszcie, dokonuje relaksacji jego sąsiadów.

Metoda `znajdz_najtansza_trase` zarządza całym procesem: losuje miasta (w razie potrzeby), buduje graf, uruchamia algorytm Dijkstry, a następnie rekonstruuje trasę od celu do startu poprzez odczyt tablicy `prev`. Zwracany jest zarówno przebieg ścieżki, jak i łączny koszt modernizacji.

Wizualizacja i weryfikacja. Klasa udostępnia szereg metod pomocniczych: `wypisz_koszty` prezentuje mapę kosztów w formie tabelarycznej, `wypisz_miasta` wypisuje pozycje startu i celu, natomiast `rysuj_trase_na_planszy` generuje wizualizację przebiegu znalezionej trasy. Metoda `weryfikacja_kosztu` umożliwia matematyczne sprawdzenie poprawności sumy kosztów ścieżki znalezionej przez algorytm.

Podsumowanie. Klasa `Bajtolandia` integruje wszystkie elementy niezbędne do rozwiązania problemu: generowanie danych, modelowanie grafu, wyszukiwanie najkrótszej

ścieżki oraz wizualizację wyniku. Jest kluczowym modułem projektu, spajającym warstwę danych z warstwą algorytmiczną.

5 Testy i weryfikacja działania

W celu usprawnienia procesu testowania algorytmu, przygotowano dedykowaną aplikację konsolową (plik wykonywalny `.exe`). Program ten umożliwia interaktywne wprowadzanie danych oraz wizualizację wyników w czasie rzeczywistym.

5.1 Interfejs użytkownika

Aplikacja komunikuje się z użytkownikiem poprzez tekstowy interfejs. Po uruchomieniu programu wyświetlane jest menu główne oraz krótka instrukcja obsługi dotycząca formatu wprowadzania danych (np. używanie spacji jako separatora zamiast przecinków).

Poniżej przedstawiono widok menu głównego aplikacji:

```
Tworzymy mapę z dwoma wyróżnionymi miastami: BAJTOGÓRA i BITOGÓRA
Chcemy znaleźć najtańszą trasę pomiędzy tymi miastami.

UWAGI PRZY WYBORZE MIAST:
- Miasta nie mogą być w tym samym miejscu.
- Miasta nie mogą ze sobą sąsiadować.
- Jeżeli wpisujesz więcej niż jedną liczbę, użyj SPACJI, nie przecinka.

=============================================
      BAJTOLANDIA - OPTYMALIZACJA TRASY
=============================================
1. Wpisz wszystkie dane ręcznie
2. Własne wymiary, ale losowe koszty
3. Wszystko losowe
4. Wyjście
```

Rysunek 1: Menu główne programu testującego

5.2 Opis trybów działania

Program oferuje trzy główne tryby pracy, pozwalające na sprawdzenie różnych aspektów działania algorytmu:

1. **Tryb ręczny (Opcja 1):**
   Ten tryb daje użytkownikowi pełną kontrolę nad strukturą problemu. Użytkownik definiuje wymiary planszy ($n \times m$), ręcznie wprowadza koszty dla każdego pola (wiersz po wierszu) oraz podaje współrzędne miast startowego i końcowego.
   Zastosowanie: Służy do weryfikacji specyficznych przypadków brzegowych (np. "ściana"z drogich pól, małe plansze 3×3) oraz testów zgodności z ręcznymi obliczeniami.

2. **Tryb pół-automatyczny (Opcja 2):**
   W tym trybie użytkownik określa jedynie wymiary planszy. Koszty modernizacji poszczególnych pól są losowane automatycznie z przedziału $[1, 9]$. Następnie użytkownik ma wybór: może ręcznie wskazać lokalizację miast lub pozwolić programowi na ich wylosowanie.
   Zastosowanie: Pozwala na szybkie testowanie wydajności i poprawności na planszach o zadanej wielkości (np. 10×10 lub 50×50), bez konieczności żmudnego wpisywania setek liczb.

3. **Tryb w pełni losowy (Opcja 3):**
   Program automatycznie losuje wszystkie parametry: wymiary planszy (w domyślnym zakresie), rozkład kosztów oraz pozycje miast.
   Zastosowanie: Służy do szybkiej weryfikacji stabilności programu i przeprowadzenia dużej liczby testów w krótkim czasie.

Po wykonaniu obliczeń program wyświetla:

- Mapę kosztów (z orientacją wiersz 0 na górze),
- Współrzędne miast,
- Znalezioną trasę (lista kroków),
- Wizualizację graficzną trasy na planszy,
- **Weryfikację matematyczną**, sumującą koszty poszczególnych pól na trasie, co pozwala upewnić się, że wynik jest poprawny.

5.3 Scenariusze testowe

Poniżej przedstawiono wyniki działania programu dla wybranych przypadków testowych.

5.3.1 Test 1: Tryb ręczny - walidacja i omijanie przeszkód

W tym scenariuszu sprawdzono odporność na błędy użytkownika oraz zdolność omijania pól o bardzo wysokim koszcie (tzw. "pułapek").

```
Wybór: 1
Liczba wierszy: 4
Liczba kolumn: 7
Wprowadzanie kosztów:
Podaj koszty dla wiersza 0 (7 liczb): 1 2 3 2 3 2 1 2
Błąd! Wymagane 7 liczb.
Podaj koszty dla wiersza 0 (7 liczb): 1 2 3 1 2 3 1
Podaj koszty dla wiersza 1 (7 liczb): 2 3 4 12 2 31 2
Podaj koszty dla wiersza 2 (7 liczb): 3 2 5 4 7 67 4
Podaj koszty dla wiersza 3 (7 liczb): 6 7 6 7 6 7 6
--- Konfiguracja Miast ---
1. Podam własne współrzędne
Wybór: 1
Podaj START (Wiersz Kolumna): 1 3
Podaj CEL (Wiersz Kolumna): 4 7
Poza zakresem! Wiersze 0-3, Kolumny 0-6.
Podaj CEL (Wiersz Kolumna): 3 6
Obliczanie najtańszej trasy...
LOKALIZACJA MIAST
Start: (1, 3) (Koszt pola: 12)
Cel:   (3, 6) (Koszt pola: 6)
--- MAPA KOSZTÓW ---
Wymiary: 4 wierszy x 7 kolumn
------------------------------------
|   1   2   3   1   2   3   1 | Wiersz 0
|   2   3   4  12   2  31   2 | Wiersz 1
|   3   2   5   4   7  67   4 | Wiersz 2
|   6   7   6   7   6   7   6 | Wiersz 3
------------------------------------
    0   1   2   3   4   5   6   (Kolumny)

>>> PRZEBIEG TRASY:
(W:1, K:3) -> (W:0, K:3) -> (W:0, K:4) -> (W:0, K:5)
-> (W:0, K:6) -> (W:1, K:6) -> (W:2, K:6) -> (W:3, K:6)

WIZUALIZACJA TRASY
```

```
(S = Start, T = Cel, # = Trasa)
-----------------------------------
|   .   .   .   #   #   #   # | Wiersz 0
|   .   .   .   S   .   .   # | Wiersz 1
|   .   .   .   .   .   .   # | Wiersz 2
|   .   .   .   .   .   .   T | Wiersz 3
-----------------------------------
    0   1   2   3   4   5   6

 KOSZT TRASY:
    Koszty pól na trasie:
    12 + 1 + 2 + 3 + 1 + 2 + 4 + 6 = 31

****************************************
*** CAŁKOWITY KOSZT TRASY: 31 ***
****************************************
```

Wnioski:

Algorytm wykazał się dużą "inteligencją"w nawigacji. Zamiast poruszać się najkrótszą drogą geometryczną, wybrał "najtańszą"trasę. Analiza mapy kosztów pokazuje, że wiersze te zawierają "korytarz"niskich kosztów (wartości 1, 2, 3, 4), podczas gdy sąsiednie rejony obfitują w drogie pola (wartości 31, 67)..
Dodatkowo algorytm idealnie poradził sobie z wprowadzonymi przez użytkownika błędnymi wartościami- zwracał komunikaty i pozwalał na ponowny wybór.

5.3.2 Test 2: Weryfikacja poprawności na losowej mapie (6×11)

W tym teście wykorzystano opcję generowania w pełni losowego środowiska. Algorytm wygenerował planszę o wymiarach 6×11.

```
Wybór: 3

Obliczanie najtańszej trasy...

LOKALIZACJA MIAST
Start: (1, 6) (Koszt pola: 8)
Cel:   (0, 2) (Koszt pola: 3)

--- MAPA KOSZTÓW ---
Wymiary: 6 wierszy x 11 kolumn
```

```
-----------------------------------------------------
|   8   5   3   8   9   7   2   2   1   5   2 | Wiersz 0
|   1   6   5   7   8   5   8   1   4   8   6 | Wiersz 1
|   9   9   2   9   1   9   8   6   9   4   6 | Wiersz 2
|   8   5   5   7   3   4   1   4   2   5   6 | Wiersz 3
|   1   9   1   6   1   6   3   6   8   8   5 | Wiersz 4
|   9   9   9   4   2   2   8   9   6   4   9 | Wiersz 5
-----------------------------------------------------
    0   1   2   3   4   5   6   7   8   9  10   (Kolumny)

>>> PRZEBIEG TRASY:
(W:1, K:6) -> (W:1, K:5) -> (W:1, K:4) -> (W:1, K:3)
-> (W:1, K:2) -> (W:0, K:2)

WIZUALIZACJA TRASY
(S = Start, T = Cel, # = Trasa)
-----------------------------------------------------
|   .   .   T   .   .   .   .   .   .   .   . | Wiersz 0
|   .   .   #   #   #   #   S   .   .   .   . | Wiersz 1
|   .   .   .   .   .   .   .   .   .   .   . | Wiersz 2
|   .   .   .   .   .   .   .   .   .   .   . | Wiersz 3
|   .   .   .   .   .   .   .   .   .   .   . | Wiersz 4
|   .   .   .   .   .   .   .   .   .   .   . | Wiersz 5
-----------------------------------------------------
    0   1   2   3   4   5   6   7   8   9  10

 KOSZT TRASY:
    Koszty pól na trasie: 8 + 5 + 8 + 7 + 5 + 3 = 36

****************************************
*** CAŁKOWITY KOSZT TRASY: 36 ***
****************************************
```

Analiza wyniku: Algorytm podjął decyzję o poruszaniu się poziomo wzdłuż Wiersza 1, zamiast natychmiastowego wejścia do wiersza 0 (gdzie znajduje się cel). Analizując mapę kosztów, widać przyczynę takiego zachowania:

- Wiersz 0 na odcinku pomiędzy startem a celem (kolumny 3, 4, 5) zawiera pola o bardzo wysokich kosztach: **8, 9, 7**.
- Wiersz 1 na tym samym odcinku oferuje koszty: **7, 8, 5**, co sumarycznie jest opcją tańszą.

Program poprawnie zidentyfikował "tańszy korytarz"w wierszu 1 i dopiero w ostatnim kroku zszedł do celu w Wierszu 0. Weryfikacja matematyczna ($8 + 5 + 8 + 7 + 5 + 3 = 36$) potwierdza poprawność obliczeń algorytmu.

5.3.3 Test 3: Omijanie bariery kosztów

W tym teście sprawdzono, czy algorytm poprawnie omija bardzo kosztowne pola i znajduje alternatywną trasę prowadzącą do celu korytarzem o niskiej cenie.

```
Wybór: 1
Liczba wierszy (Wysokość): 5
Liczba kolumn (Szerokość): 5
Wprowadzanie kosztów (Wiersz 0 to GÓRA):
Podaj koszty dla wiersza 0 (5 liczb): 1 1 99 99 1
Podaj koszty dla wiersza 1 (5 liczb): 1 1 99 99 1
Podaj koszty dla wiersza 2 (5 liczb): 1 1 99 99 1
Podaj koszty dla wiersza 3 (5 liczb): 1 1 1 1 1
Podaj koszty dla wiersza 4 (5 liczb): 1 1 1 1 1
--- Konfiguracja Miast ---
1. Podam współrzędne
2. Wylosuj
Wybór: 1
Podaj START (Wiersz Kolumna): 0 1
Podaj CEL (Wiersz Kolumna): 0 4
Obliczanie najtańszej trasy...
--- LOKALIZACJA MIAST ---
Start: (0, 1) (Koszt pola: 1)
Cel:   (0, 4) (Koszt pola: 1)
--- MAPA KOSZTÓW ---
Wymiary: 5 wierszy x 5 kolumn
(Wiersz 0 jest na GÓRZE, Wiersz 4 na DOLE)
----------------------------
|   1   1  99  99   1 | Wiersz 0
|   1   1  99  99   1 | Wiersz 1
|   1   1  99  99   1 | Wiersz 2
|   1   1   1   1   1 | Wiersz 3
|   1   1   1   1   1 | Wiersz 4
----------------------------
    0   1   2   3   4   (Kolumny)

>>> PRZEBIEG TRASY (W=Wiersz, K=Kolumna):
```

```
(W:0, K:1) -> (W:1, K:1) -> (W:2, K:1) -> (W:3, K:1)
-> (W:3, K:2) -> (W:3, K:3) -> (W:3, K:4)
-> (W:2, K:4) -> (W:1, K:4) -> (W:0, K:4)

--- WIZUALIZACJA TRASY ---
(S = Start, T = Cel, # = Trasa)
----------------------------
|   .   S   .   .   T | Wiersz 0
|   .   #   .   .   # | Wiersz 1
|   .   #   .   .   # | Wiersz 2
|   .   #   #   #   # | Wiersz 3
|   .   .   .   .   . | Wiersz 4
----------------------------
    0   1   2   3   4

>>> WERYFIKACJA MATEMATYCZNA KOSZTU:
    Koszty pól na trasie: 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 10
    Status: OK

****************************************
*** CAŁKOWITY KOSZT TRASY: 10 ***
****************************************
```

Analiza wyniku: Mimo że start i cel znajdowały się (geometrycznie) blisko, algorytm wybrał tańszą trasę omijając pola z wysokim kosztem.

5.4 Wnioski z testów

Przeprowadzone testy potwierdziły poprawność działania zaimplementowanego algorytmu Dijkstry.

- Program poprawnie identyfikuje najtańsze ścieżki, nawet jeśli wymagają one nadłożenia drogi
- Mechanizmy walidacji poprawnie blokują błędne dane wejściowe
- Weryfikacja matematyczna na końcu działania programu (sumowanie kosztów pól) jest zgodna z wynikiem zwróconym przez funkcję kosztu.

6 Podsumowanie i wnioski

W ramach projektu udało się stworzyć w pełni funkcjonalne narzędzie do planowania optymalnych tras modernizacji dróg w Bajtolandii.

Główne osiągnięcia:

- **Poprawność:** Algorytm bezbłędnie znajduje najtańszą ścieżkę, co potwierdzono w testach manualnych i losowych.
- **Elastyczność:** Implementacja jest modułowa. Łatwo można by ją rozszerzyć o nowe funkcjonalności, np. różne rodzaje terenu, teleporty czy blokady dróg (koszt nieskończony).
- **Wizualizacja:** Prosty, ale czytelny interfejs tekstowy pozwala na szybką weryfikację wyników.

7 Podział pracy w zespole

Realizacja projektu była efektem ścisłej współpracy obu członków zespołu. Zadania zostały podzielone w sposób umożliwiający równoległą pracę, przy zachowaniu ciągłej komunikacji.

Karolina Gralak:

- Opracowanie koncepcji klasy `Graph`.
- Implementacja algorytmu Dijkstry
- Przeprowadzenie testów
- Przygotowanie raportu w systemie LaTeX

Wiktor Chmiel:

- Implementacja klasy `Bajtolandia` (generowanie mapy, losowanie miast).
- Przygotowanie wizualizacji konsolowej
- Przygotowanie raportu w systemie LaTeX .

Literatura

[1] Lech Banachowski, Krzysztof Diks, and Wojciech Rytter. *Algorytmy i struktury danych.* WNT, Warszawa, 2003.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Wprowadzenie do algorytmów.* Wydawnictwo Naukowe PWN, Warszawa, 2012.

[3] Python Software Foundation. *Dokumentacja języka Python 3.14.* `https://docs.python.org/3/`.

[4] Anna Radzikowska. Materiały wykładowe z przedmiotu algorytmy i struktury danych. Semestr zimowy 2025/2026.